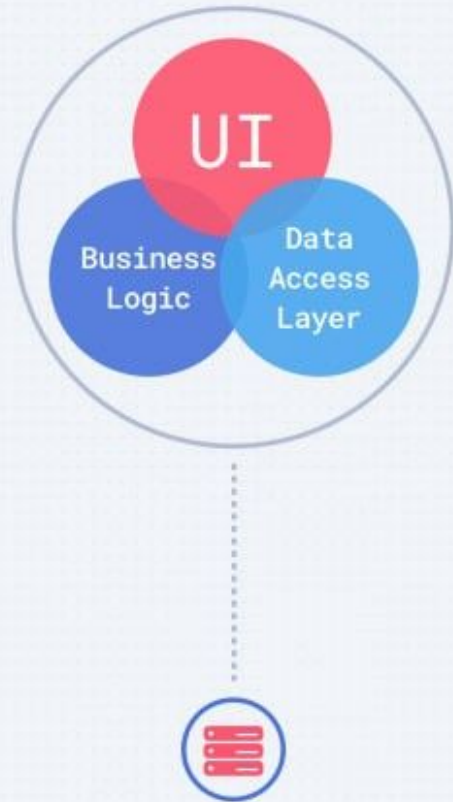
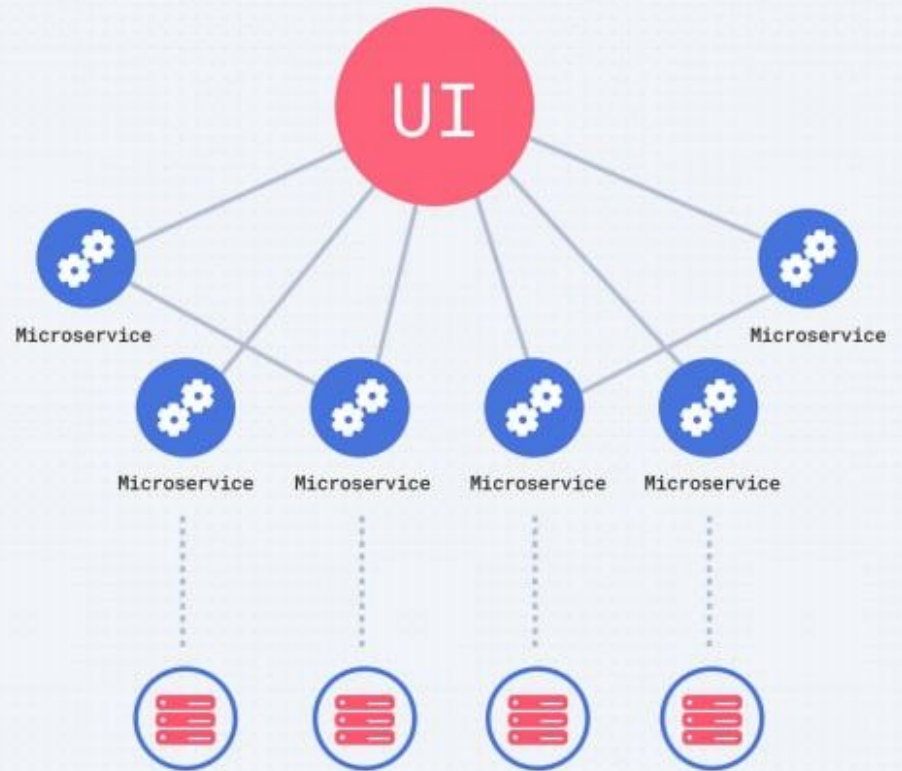

Микросервисная архитектура

— организация общения —

Monolithic Architecture



Microservices Architecture



Тип протокола

Синхронный протокол. При отправке запроса клиент ожидает ответ от службы — задачи выполняются только после того, как сервер пришлет ответ. Пример такого протокола — HTTP.

Асинхронный протокол. Код клиента или отправители сообщения не ждут ответ — сообщения отправляются аналогично передаче в очередь любого брокера. Такой тип применяет протокол AMQP.

Синхронное взаимодействие (Synchronous)

Клиент отправляет запрос и ждет ответа перед тем, как продолжить работу. Если сервис-получатель не отвечает, клиент блокируется или выдает ошибку.

Основные протоколы:

- HTTP/REST (JSON)
- gRPC (HTTP/2 + Protobuf)
- GraphQL

HTTP/REST (JSON)

Как работает: Стандартные HTTP-методы (GET, POST, PUT, DELETE). Данные в формате JSON.

Плюсы: Простота, огромная поддержка инструментов, человекочитаемость.

Минусы: Текстовый формат (тяжелее бинарного), слабая типизация (нужен Swagger/OpenAPI), оверхед заголовков.

Когда использовать: Внешние API, публичные интерфейсы, простые внутренние вызовы.

gRPC (HTTP/2 + Protobuf)

Как работает: Бинарный протокол на основе HTTP/2. Использует Protocol Buffers для сериализации.

Плюсы: Очень высокая скорость, строгая типизация (контракты), поддержка потоковой передачи (streaming).

Минусы: Сложнее в отладке (бинарный формат), меньше поддержка в браузерах (нужен прокси).

Когда использовать: Внутреннее общение между сервисами (Service-to-Service), высоконагруженные системы.

GraphQL

Как работает: Клиент запрашивает только те данные, которые ему нужны, через единый эндпоинт.

Плюсы: Гибкость, отсутствие проблем недо/перегрузки данными (over/under-fetching).

Минусы: Сложность кэширования, риск сложных запросов (DoS), сложнее в реализации сервера.

Когда использовать: Когда у клиентов (фронтенд/мобайл) очень разные требования к данным.

Асинхронное взаимодействие (Asynchronous)

Клиент отправляет сообщение и не ждет немедленного ответа. Сервис-получатель обработает сообщение, когда сможет. Это снижает связанность (coupling).

Основные механизмы:

Очереди сообщений (Message Queues):

- Инструменты: RabbitMQ, ActiveMQ, AWS SQS.
- Паттерн: Point-to-Point. Сообщение потребляется только одним сервисом.
- Зачем: Выравнивание нагрузки (buffer), гарантия доставки, надежная обработка фоновых задач.

Шина событий / Стриминг (Event Bus / Streaming):

- Инструменты: Apache Kafka, AWS Kinesis, Redis Streams.
- Паттерн: Pub/Sub (Publish-Subscribe). Сообщение может быть прочитано многими подписчиками.
- Зачем: Реактивная архитектура, Event Sourcing, передача потоков данных в реальном времени, интеграция разных контекстов.

Важные инфраструктурные паттерны

API Gateway (Шлюз):

- Единая точка входа для внешних клиентов.
- Занимается аутентификацией, ограничением частоты запросов (rate limiting), маршрутизацией и агрегацией ответов от разных сервисов.
- Примеры: Kong, Nginx, Spring Cloud Gateway, AWS API Gateway.

Service Discovery (Обнаружение сервисов):

- В динамической среде (Kubernetes, Docker) IP-адреса сервисов меняются. Сервисы не должны хардкодить адреса друг друга.
- Сервис регистрируется при старте, клиент спрашивает у реестра, где найти нужный сервис.
- Примеры: Consul, Eureka, Kubernetes DNS (CoreDNS).

Saga (Сага):

- Паттерн для управления распределенными транзакциями.
- Если в цепочке сервисов один падает, выполняются компенсирующие действия (откат изменений) в предыдущих сервисах.

Service Mesh (Сервисная сетка):

- Выносит логику общения (балансировка, retries, шифрование, метрики) из кода приложения в отдельный слой (sidecar-прокси).
- Примеры: Istio, Linkerd.

Практика

1. Feign client

2. WebClient

3. RestTemplate

Что почитать:

[Взаимодействие в архитектуре микросервисов / Хабр](#)

[Microservices. Как правильно делать и когда применять?](#)

[26 основных паттернов микросервисной разработки](#)